EXP 6 : ERROR CORRECTION AT DATA LINK LAYER

AIM:

To write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

ERROR CORRECTION AT DATA LINK LAYER:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

CREATE SENDER PROGRAM WITH BELOW FEATURES:

1. Input to sender file should be a text of any length. Program should convert the text to binary.

2. Apply hamming code concept on the binary data and add redundant bits to it.

3. Save this output in a file called channel.

CREATE A RECEIVER PROGRAM WITH BELOW FEATURES:

1. Receiver program should read the input from Channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

STUDENT OBSERVATION:

CODE: (Python)

```
m = int(input("Enter number of data bits:"))
data = input("Enter data bits : ")
#Find the no. of redundant bits
r=0
while 2**r < m+r+1:
    r+=1
n = m+r
print("Number of redundant bits : ", r)
h = [0] * (n+1)  #Create code
j = 0  # Put data bits
for i in range(1, n+1):
    if i not in [1,2,4,8,16]:
```

```
            h[i] = data[j]
            j += 1

    for p in [1, 2, 4, 8, 16]:
        if (i & (i-1)) != 0:
            h[i] = data[j]
            j += 1

#Calculate redundant bits
    for i in range(r):
        p = 2**i
        count = 0

        for j in range(1, n+1):
            if j & p:
                if h[i] == '1':
                    count += 1

        if count % 2 == 1:
            h[p] = '1'
        else:
            h[p] = '0'

#Print redundant bits as normal string redundant.
    for i in range(r):
        redundant += h[2**i]

    print("Redundant bits: ", redundant)
```

```
#Print transmitted data as normal string transmitted = ""
    for i in range (1, n+1):
        transmitted += h[i]
    print("Data to be transmitted :", transmitted)

#Ask for error.
    choice = input("Is any transmitted bit changed?
                    (yes/no) :").lower()

    if choice == "no":
        received = input("Enter the transmitted bit:")
    else:
        received = input("Enter the transmitted bit:")
        index = int(input("Enter the index to be
                           changed :"))
        received = list(received)

#Change the selected bit
        if received[index-1] == '0':
            received[index-1] = '1'
        else:
            received[index-1] = '0'
        received = ''.join received
        print("Received code :", received)

#Find error position
        error_position = 0
        for i in range(r):
            p = 2**i
```

```
        count = 0
        for j in range(1, n+1):
            if j & p:
                if received[j-1] == '1':
                    count += 1
        if count % 2 == 1:
            error_position += p
# Correct the code
    if error_position == 0:
        print("No error detected")
        print("Corrected code:", received)
    else:
        print("Error detected at index:", error_position)
        received = list(received)
        if received[error_position-1] == '0':
            received[error_position-1] == '1'
        else:
            received[error_position-1] = '0'
        received = ''.join(received)
        print("Corrected code:", received)
```

OUTPUT:

```
Enter the number of data bits : 7
Enter data bits : 1001101
Number of redundant bits : 4
Redundant bits : 0110
Data to be transmitted : 01110010101
Is any transmitted bit changed ?(yes/no) : yes
Enter the transmitted bit : 01110010101
Enter the index to be changed : 7
Recaived code : 01110000101
Error detected at index : 7
Corrected code : 01110010101
```

RESULT:

Thus, the implementation of hamming code for error detection and error corrected is executed and verified succesfully.